

All On-Board: Fully On-Chip Neuromorphic Q-Learning with Embedded CartPole Simulation

Steven C. Nesbit^{*†}, Giovanni T. Michel^{†‡}, Gerd J. Kunde[§], Edward Kim^{*}, and Andrew T. Sornborger[†]

^{*}College of Computing and Informatics, Drexel University, Philadelphia, PA, USA

[†]Information Sciences (CAI-3), Los Alamos National Laboratory, Los Alamos, NM, USA

[‡]Department of Electrical and Computer Engineering, Northwestern University, Evanston, IL, USA

[§]Nuclear & Particle Physics & Applications (P-3), Los Alamos National Laboratory, Los Alamos, NM, USA

Email: [†]nesbitsc@lanl.gov

Abstract—As AI models grow in size and usage, their energy demands increase dramatically, raising sustainability and economic concerns. Neuromorphic hardware, inspired by the energy efficiency of the brain, seeks to address this challenge by offering low-power, fast-processing alternatives to conventional computing. Such hardware is particularly well-suited to control systems deployed in resource-constrained environments, which are best trained via reinforcement learning (RL). This contribution presents the design and implementation of a fully on-chip, closed-loop Loihi 2 RL agent. Our neuromorphic circuit consists of a fully embedded Q-learning algorithm and an on-chip simulation of the CartPole-v0 environment on Loihi 2. Our Q-learning algorithm trained the same number of successful agents as the CPU implementation in only half the execution time and with two orders of magnitude less dynamic power. These findings demonstrate the viability of RL on neuromorphic hardware and highlight its promise for building energy-efficient, real-time, embedded AI systems.

Index Terms—neuromorphic hardware, reinforcement learning, online training

I. INTRODUCTION

A. Energy Efficiency in Artificial Intelligence

The growing demand for AI applications has brought about Nobel Prize-winning advancements across various domains such as physics and chemistry [1]. However, these advancements come with increasing computational costs, particularly in terms of energy consumption. Energy efficiency in AI models has become an urgent concern, driven by the growing environmental impact of large-scale machine learning (ML) systems and the economic implications of high-power computational infrastructure [2], [3]. As AI continues to scale, optimizing energy consumption is crucial for both economic sustainability and mitigating the environmental burden [4]–[7].

One of the key strategies for addressing the energy challenges in AI is the development of energy-efficient algorithms and architectures. Research in this area has focused on reducing the number of computations required for both training and inference while maintaining or improving model performance [8]. Several approaches have been proposed to achieve this

goal, including model compression, quantization, pruning, and the development of specialized hardware accelerators [9], [10].

The design of energy-efficient hardware accelerators is a critical area of research. GPUs and TPUs have been widely adopted for AI workloads due to their ability to perform parallelized computations [10]. However, neuromorphic hardware, which mimics the brain’s event-driven processing capabilities, offers a promising path forward for energy-efficient AI. Neuromorphic systems, such as Intel’s Loihi 2 chip, leverage spiking neural networks (SNNs) to process information in a manner that closely resembles the human brain [11]–[13]. SNNs operate on discrete events (spikes), which means that computations only occur when a spike is generated, significantly reducing energy consumption [12], [14], while also reducing power density. For instance, the IBM TrueNorth neuromorphic chip power density was 20 mW/cm² as opposed to typical CPU power densities of 50 – 100 W/cm² [15]. Moreover, neuromorphic systems are inherently parallel, allowing them to perform many computations simultaneously with minimal power consumption. This makes them ideal for tasks where low latency and energy efficiency are critical [12], [16].

Despite these advantages, a critical gap remains: implementing complete learning algorithms on neuromorphic hardware without external computational support. Hardware-specific constraints, including limited on-chip plasticity rules, local-only memory access, asynchronous execution, and reduced numerical precision, make on-chip learning algorithms challenging to implement. Exact implementations of conventional learning algorithms remain uncommon; neuromorphic back-propagation has demonstrated that a conventional gradient-based learning algorithm can be executed entirely on Intel’s Loihi neuromorphic processor using synfire-gated control of information propagation and learning [17], [18].

Neuromorphic reinforcement learning has nevertheless been explored through several complementary approaches. Spiking actor-critic networks have solved simulated control tasks including CartPole [19], while memristive systems have demonstrated or modeled in-situ and online reinforcement learning for CartPole and related benchmarks [20]–[22]. On Intel Loihi, prior work deployed reinforcement-learning policies or value networks for low-power control, including spiking DDPG for robotic navigation and deep spiking Q-networks for CartPole

This work was supported by the US Department of Energy National Nuclear Security Administration’s Office of Defense Nuclear Nonproliferation Research & Development (DNN R&D) at Los Alamos National Laboratory under contract 89233218CNA000001.

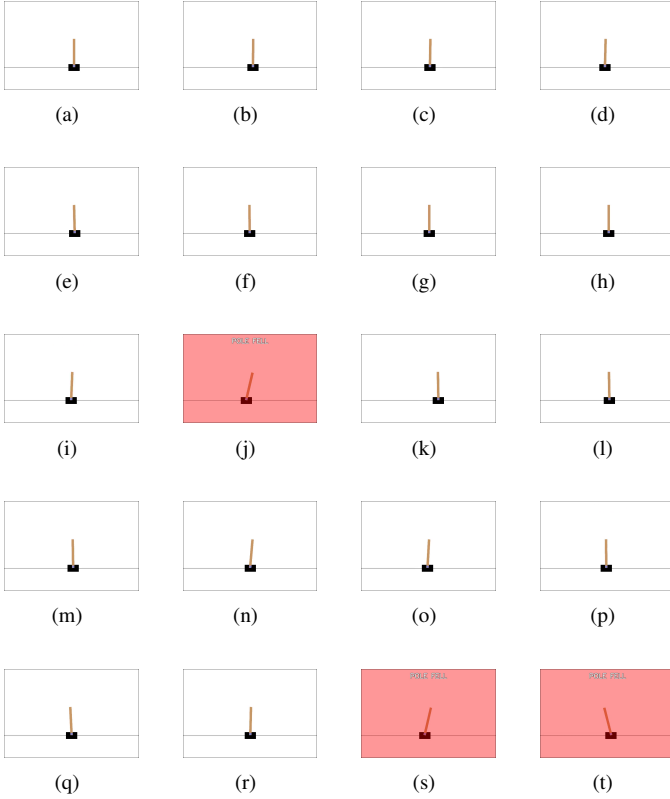


Fig. 1. **Visualizations of CartPole environment.** Corresponding 30-second videos are available at <https://github.com/nasbits/all-on-board-figure-1>. **a:** OpenAI Gym environment inference using agent with ideal Q-table, **b:** Loihi 2 environment inference using agent with ideal Q-table, **c:** OpenAI Gym environment inference using on-chip trained agent, **d:** Loihi 2 environment inference using same agent as (c), **e:** OpenAI Gym environment inference using on-chip trained agent, unique from (c) and (d), **f:** Loihi 2 environment inference using same agent as (e), **g:** OpenAI Gym environment inference using CPU-trained agent, **h:** OpenAI Gym environment inference using CPU-trained agent, unique from (g), **i:** Loihi 2 environment inference using on-chip-trained agent trained for 0, **j:** 10 000, **k:** 20 000, **l:** 30 000, **m:** 40 000 weight updates, **n:** Loihi 2 environment training at 0, **o:** 10 000, **p:** 20 000, **q:** 30 000, **r:** 40 000 weight updates, **s:** Loihi 2 environment inference using exploration rate decay function beginning at 30% and **t:** 10% randomness.

and Acrobot [23]–[25]; related neuromorphic control systems have also demonstrated on-chip adaptation outside the Q-learning setting [26]. More recent work has examined directly trained spiking RL on neuromorphic accelerators, event-based neuromorphic RL architectures, and hardware-feasible spike-based Q-learning [27]–[29]. These studies establish the feasibility of energy-efficient neuromorphic control and learning, but differ from the present implementation in computational boundary: here, the Q-learning update, action selection, reward/state handling, and the CartPole environment simulation are co-located on Loihi 2 so that the closed training loop does not require host computation between environment transitions.

B. Q-Learning

One type of algorithm that can be used to train neuromorphic systems for real-time control tasks is Q-learning; a foundational reinforcement learning (RL) algorithm intro-

duced by Watkins in 1989 [30]. Q-learning is a model-free unsupervised learning method that allows an agent to learn the optimal policy for a given environment by interacting with it and maximizing cumulative rewards. Q-learning’s compatibility with neuromorphic hardware stems from its discrete state–action space and iterative weight updates, which can be naturally mapped to spike-based neural dynamics. Q-learning has been applied successfully to a wide range of applications, particularly in robotics [31], [32], autonomous vehicles [33], [34], and smart grid systems [35], [36]. These applications share a common requirement: real-time learning in resource-constrained environments, where neuromorphic efficiency advantages are most critical.

$$Q^{\text{new}}(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (1)$$

The Q-learning action–value function $Q(s_t, a_t)$, which estimates the expected cumulative reward that can be obtained by taking action a in state s at time t , is updated via Equation 1, which may be broken down into the following components:

$$TD_{\text{target}} = r_{t+1} + \gamma \max_a Q(s_{t+1}, a) \quad (2)$$

$$TD_{\text{error}} = TD_{\text{target}} - Q(s_t, a_t) \quad (3)$$

$$U = \alpha TD_{\text{error}} \quad (4)$$

$$Q^{\text{new}}(s_t, a_t) \leftarrow Q(s_t, a_t) + U \quad (5)$$

Time step t refers to the time at which an action was taken rather than the algorithmic time steps described in Section II-B. The parameter α is the learning rate, r_{t+1} is the immediate reward received after taking action a_t in state s_t , and γ is the discount factor, which determines the importance of future rewards compared to immediate rewards. The term $\max_a Q(s_{t+1}, a)$ is the maximum Q-value for the next state s_{t+1} , representing the best possible outcome from that state onward. TD_{target} is the temporal difference target (an estimate of the future return), TD_{error} is to the temporal difference error (the difference between the agent’s current estimate of a state–action value and a more accurate estimate based on observed experience), and U is the Q-value update term.

C. CartPole Environment

The CartPole problem is a classical control task widely used as a benchmark for testing the effectiveness of RL algorithms such as Q-learning [37], [38]. It was developed by Michie and Chambers in 1968 [39] and has become a fundamental task in the field of machine learning and adaptive control systems. The problem consists of balancing a pole on a moving cart by applying forces to the cart in either direction. The objective is to prevent the pole from falling by keeping it upright for as long as possible, which is achieved by controlling the

horizontal movement of the cart. Recent work has formalized CartPole specifically as a neuromorphic-computing benchmark and proposed progressively more difficult variants, noting that the standard formulation provides a useful starting point but is not by itself a particularly challenging test of modern AI agents [40]. We intentionally retain the standard CartPole-v0 formulation here because the objective is not to establish state-of-the-art CartPole control performance, but to evaluate whether the complete reinforcement-learning loop, including the learning rule, action selection, reward processing, and nonlinear environment dynamics, can execute on a single neuromorphic processor.

The environment's dynamics, governed by classical mechanics equations involving trigonometric functions and divisions, present a cart that moves along a horizontal track, with a pole that pivots freely along the cart's center. The state of the system can be described by two variables: the position, x , of the cart along the track and the angle, θ , of the pole relative to the vertical axis. The control input to the system is the horizontal force applied to the cart. This force causes the cart to accelerate, which in turn affects the motion of the pole. The goal is to control the system such that the pole remains balanced while the cart remains within predefined limits on the track. The challenge arises from the fact that small deviations in θ can quickly lead to instability, requiring rapid corrective actions to prevent the pole from falling.

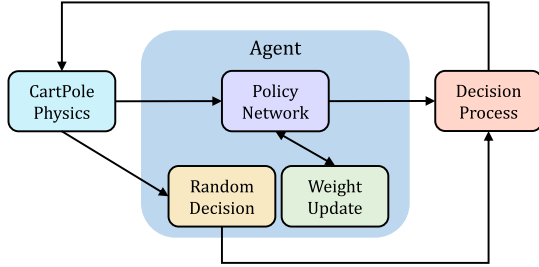


Fig. 2. **Overview of neuromorphic Q-learning circuit architecture.** Angle and angular velocity measurements are sent from the CartPole neuron to the policy network, which sends these values through the Q-table. Q-table output from the policy network goes to the decision process module, which determines whether or not to take a left or right action. The CartPole neuron also sends reward information to the random decision module, which decides whether (exploration) or not (exploitation) to select an action randomly, and if so, makes a random decision and overrides the greedy decision made by the decision process module. The policy network also sends its Q-table output values to the weight update module, which in turn sends Q-value update information to the policy network. Finally, the action selected by the decision process module is sent to the CartPole neuron to calculate the next state of the CartPole environment.

II. METHODS

A. Neuromorphic CartPole Simulation

To enable fully on-chip execution of Q-learning and inference for the CartPole environment, we developed a neuromorphic simulation of OpenAI Gym's CartPole-v0 for deployment on the Loihi 2 neuromorphic processor utilizing Lava on Loihi microcode. This simulation was adapted from the original OpenAI Gym CartPole source code available on GitHub [41].

$$\ddot{\theta} = \frac{g \sin \theta + \cos \theta \left(-F - m_p l \dot{\theta}^2 \sin \theta \right) / (m_c + m_p)}{l \left(\frac{4}{3} - \frac{m_p \cos^2 \theta}{m_c + m_p} \right)} \quad (6)$$

$$\ddot{x} = \frac{F + m_p l \left(\dot{\theta}^2 \sin \theta - \ddot{\theta} \cos \theta \right)}{m_c + m_p} \quad (7)$$

Equations 6 and 7 governing the dynamics of the CartPole system, as outlined by [42], describe the relationship between the applied force, F (± 10 N), the acceleration, $\ddot{x}$, of the cart of mass m_c (1 kg), and the angular acceleration, $\ddot{\theta}$, of the pole of mass m_p (0.1 kg) and length $2l$ (1 m). CartPole serves as an ideal testbed because: (1) it is a standard RL benchmark enabling direct comparison, (2) its physics simulation can be embedded on-chip, and (3) its discrete action space aligns well with neuromorphic architectures.

1) *Function Approximation:* The calculations for CartPole dynamics, expressed in Equations 6 and 7, had to be implemented using only a limited microcode instruction set. Notably, the sine and cosine functions were not directly supported and therefore required a custom implementation. The sine function was approximated within the range of $[-0.418, 0.418]$ rad (the maximum possible range of the pole angle) by $y = 0.98214x$ with a maximum absolute error of 0.00229. The cosine function was approximated within this same range with the function $y = -0.00012052x^2 + 4096$ with a maximum absolute error of 0.00020. In addition, Newton's method was employed to iteratively approximate division by finding the roots of the reciprocal function, $\frac{1}{b}$, where b is the divisor, and then multiplying the result by the numerator, giving a maximum absolute error of 0.00291. These calculation errors combine to give a maximum error in $\ddot{\theta}$ of 0.03632 rad/s² and a maximum error in $\ddot{x}$ of 0.00166 m/s².

2) *Random Number Generation:* In the OpenAI Gym CartPole environment, random numbers are used to initialize the state variables—cart position, cart velocity, pole angle, and pole angular velocity—uniformly within the range $[-0.05, 0.05]$. These values are generated using the `numpy.random.uniform()` function, which relies on the Mersenne Twister MT19937 algorithm [43] and is seeded with nondeterministic data from the operating system [44], [45]. Although replicating the MT19937 algorithm would have provided an exact match to the Python CartPole implementation, it was infeasible to incorporate within the CartPole process on Loihi 2 due to hardware constraints. To address this, a simpler linear congruential generator (LCG) [46] was chosen and is defined by:

$$X_{n+1} = (aX_n + c) \bmod m \quad (8)$$

where X_n is the current seed, a is the multiplier, c is the increment, and m is the modulus. To generate pseudo-random 23-bit unsigned integers, the constants $a = 65793$, $c = 4282663$, and $m = 2^{23}$ were used, following the LCG implementation in the `cc65` development package [47],

[48]. While the LCG produces lower-quality random numbers (period of 2^{23} vs. 2^{19937}), the random number generator was only called a maximum of 400 000 times by the CartPole process per training run.

3) *CartPole State Variables*: In addition to computing cart acceleration and pole angular acceleration, the neuromorphic CartPole simulation maintained and updated four key state variables governing the environment’s physics: cart position, cart velocity, pole angle, and pole angular velocity. These variables were updated at each simulation step using the Euler method, with a fixed time step duration of 0.02 s. After each update, the simulation checked if the environment had entered a failure state. A failure state was triggered if the cart position exceeded ± 2.4 m from the center or if the pole angle magnitude surpassed 0.2095 rad. At each time step, the CartPole process output five variables as 24-bit signed integers: pole angle, pole angular velocity, reward, cart position, and cart velocity. Comparisons between neuromorphic and CPU CartPole environments can be seen in Figs. 1(a) through 1(f). Further visualizations, described in the caption and below may be seen in Figs 1(g) through 1(t).

B. Q-Learning

An overview of the neuromorphic Q-learning circuit is shown in Fig. 2. A detailed schematic of functional connectivity is provided in Fig. 3, with hyperparameters outlined in Table I. Our neuromorphic Q-learning circuit consists of five interconnected modules, synchronized by a gating chain that controls the nine-step algorithmic loop summarized as follows:

- **T₀**: The CartPole simulator is denoted as layer *Cp* and updates its state (position, velocity, pole angle, and angular velocity) based on the previous iteration (at the beginning of an epoch, it is set to a random state). *Cp* gates its angle and angular velocity values to layer *S_{t+1}* (this value persists in the layer and is used at **T₆**) and to layer *Ip* (the input to the Q-table). *Cp* gates the reward value (+1 for a balanced pole and in-bounds cart, or −1 otherwise) to rate calculator layer *Rc* and a relay layer, *Rw₀*. The first neuron in the gating chain triggers processing in **T₁** (and similarly downstream).
- **T₁**: The input layer encodes the state as a one-hot vector of length 72 and sends it through a weight matrix (Q-table) to the output layer, *Op*. Simultaneously, the rate calculation process *Rc* uses the reward to compute the episode score and derives the exploration rate ϵ and learning rate α using Equation 11. These values are then sent to the random action process, *Ra*, and a relay neuron, α_0 , respectively. The episode score is also used to determine whether or not to terminate training early. If an episode score of 500 is reached, *Rc* will emit an inhibitory signal to the gating chain at neuron *G₂* to halt the execution of the synfire gating chain, and thus end training. Additionally, the relay value from *Rw₀* is gated to *Rw₁* at this step.
- **T₂**: The output layer, *Op*, (of size two, corresponding to left and right actions) then sends the Q-values for state–

action pair (s_{t+1}, a) to the action selection layer, *As*, the target layer, *Tg*, and a relay neuron, *O_{RO}*, that stores $Q(s_t, a_t)$ for the next loop’s weight update. *Rw₁* gates the reward (scaled by a weight of 8, yielding values of −8 or +8) to target layer *Tg*. The random action process can also send an override signal to the action selection layer, *As*, to initiate a random action. Relay neuron α_0 gates the learning rate α to layer α_1 .

- **T₃**: The selected action a_{t+1} stored in *As* is gated to a relay layer, *AR*, for later use by the CartPole layer, *Cp*, and is gated to *A_t* for use in the next iteration. *A_t*, persistent from the previous iteration, is gated to the update calculator layer, *Up*. Layer *Tg* computes TD_{target} from its input via Equation 2 and gates it to *Up*. Layer α_1 gates α to layer *Up*, and the Q-value at *t* is gated from *O_t* to layer *Up*. *O_t* contains persistent policy output from the previous iteration. Simultaneously, *O_{RO}* is gated to *O_{R1}*.
- **T₄**: The update calculator, *Up*, computes $\alpha(TD_{\text{error}})$ (see Equations 3 and 4) from its inputs and gates it to weight update trace layer, *Wu*. In parallel, *O_t* is gated to *Wu*.
- **T₅**: The weight update trace layer, *Wu*, determines the Q-value update, y_1 , which is gated to *Op*.
- **T₆**: The persistent state in relay neuron *S_t* is updated and the previous state is sent to the input layer, *Ip*.
- **T₇**: The input layer, *Ip*, sends s_t to the output layer, *Op*.
- **T₈**: Via Hebbian learning, the Q-values associated with the current state are updated using the update trace, y_1 . The stored action a_{t+1} is gated as input to the CartPole simulation. Meanwhile, synfire gating chain neuron *G₈* sends a signal to *G₀*, restarting the sequence for the next state–action cycle.

TABLE I
HYPERPARAMETERS USED IN NEUROMORPHIC Q-LEARNING.

Hyperparameter	Value
Q-value update function	Equation 1
α , ϵ decay function	Equation 11
reward r_{t+1}	± 8
discount factor γ	0.99
Q-table shape	2×72
Q-value initialization	−127
Q-value range	[−127, 126]
termination condition	score = 500

The modules and custom neuron processes shown in Figs. 2 and 3 are described in detail for the remainder of this section.

1) *Gating Chain*: Depicted at the top of Fig. 3, the gating chain serves a similar role to that described in neuromorphic backpropagation [17]. The gating chain fires in a continuous loop, with each neuron activating the next in sequence, controlling the propagation of information throughout the network. A total of nine gating neurons are used, corresponding to the nine-time step loop of the Q-learning algorithm.

2) *Policy Network*: The policy network consists of three unique processes, the first of which is the input layer, *Ip*. The input layer is responsible for receiving pole angle and pole angular velocity information from the CartPole process, *Cp*,

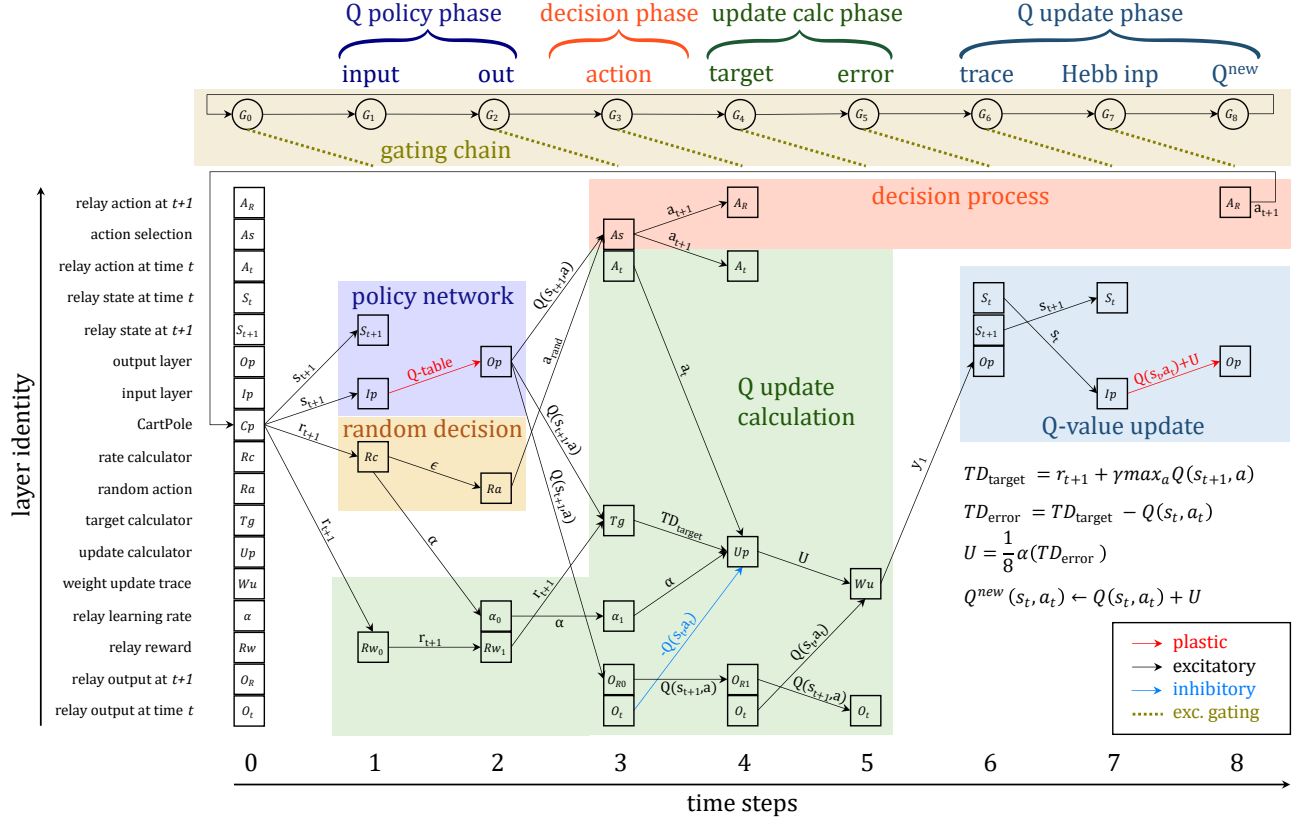


Fig. 3. **Functional connectivity of the neuromorphic Q-learning circuit.** Layers are only shown when gated ‘on,’ and synapses are only shown when their target is gated on. The names of the neuron layers are given on the left margin so that the row corresponds to layer identity. The columns correspond to the time steps of the algorithm, which correlate to the time steps on Loihi 2. Arrow labels indicate information flow through the synapse. Variables ϵ and α refer to the exploration and learning rates, respectively (see Equation 11). a_{rand} , random action; y_1 , constrained update term (see Sections II-B2 and II-B5).

and converts this into a one-hot-encoded state vector. This is achieved by discretizing the pole angle into six bins and the pole angular velocity into twelve bins. This discretization was based on empirical testing of quantized off-chip Q-learning. The input layer contains 72 neurons, each corresponding to a unique combination of threshold values for pole angle and pole angular velocity. This 72-state representation balances state space coverage with neuromorphic memory limitations.

The output layer, Op , consists of two neurons that are connected to the input layer via a plastic connection with a weight matrix $Q \in \mathbb{R}^{2 \times 72}$, which serves as the Q-table. With the input vector having a value of 1 for the neuron representing the current state, the resulting output vector $q = sQ^\top$ produces both Q-values corresponding to that state. The output layer also receives input indicating how to update the plastic connection. We implement Q-table updates using Hebbian learning, where the tag variable T tracks the target weight change, and the weight update dW applies this change only to the active state-action pair. Specifically:

$$dT = 2u_0y_1 - u_0T \quad (9)$$

$$dW = x_0T \quad (10)$$

In Equation 9, T represents a register in the Loihi 2 learning engine referred to as the tag variable. This 8-bit signed register is updated before the 8-bit signed weight variable W , which is updated according to Equation 10. The variable u in Equation 9 denotes how frequently to read from specific registers, which is every time step in this case. The variable y_1 in Equation 9 dictates the value of the weight change. The term $-u_0T$ ensures that the tag variable is always equal to the value of the weight update trace y_1 . In Equation 10, the variable x_0 represents the spike count from input neurons that had sent activity through the plastic connection. This register is multiplied by the tag variable, and the result is bit-shifted right by 1 bit to yield the final weight update dW .

The third type of microcoded neuron process used in the policy network is the relay neuron, S_{t+1} , which serves as a neuromorphic memory register with data accessible to other neurons. Here it is used to store state information at time $t+1$ for use in Hebbian weight updates, as shown in Fig. 3.

3) *Random Decision:* The Q-learning algorithm relies on random action selection to explore the state space and determine which actions yield higher cumulative rewards for given states. The probability of overriding the Q-policy decision with a random choice is governed by Equation 11, as used by [49].

The output of this equation is also used as the learning rate in the weight update module. The random decision module in the neuromorphic Q-learning algorithm is responsible for determining when to take a random action, selecting which random action to take, and calculating the learning rate to be used by the weight update module.

$$\epsilon, \alpha = \max \left(0.1, \min \left(1, 1.0 - \log_{10} \left(\frac{\text{score} + 1}{25} \right) \right) \right) \quad (11)$$

The rate calculator process, *Rc*, receives input from the CartPole simulation's reward value and accumulates the number of valid states as a total score for each episode in the CartPole environment, which serves as input to Equation 11. The resulting exploration rate ϵ is sent from the rate calculator neuron to the random action neuron, *Ra*, so that it can determine the probability of selecting a random action and, subsequently, which action to take. The random action neuron employs the same LCG as the CartPole process. If the random number is greater than the ϵ input, no random action is selected, otherwise a random value of -1 or 1 is sent as output.

4) *Decision Process*: The decision process module consists of one unique process, the action selection neuron, *As*, as well as a gated relay neuron, *Ar*, used to store the value sent by the action selection neuron. The action selection neuron receives input from the output layer of the policy network. The weight of the connection from the first neuron of the output layer to the action selection process is -1 , and the weight of the connection from the second neuron is 1 . These values are summed within the action selection neuron's dendrite accumulator. Therefore, a larger Q-value for the left action results in a negative spiking input to the action selection neuron, a larger Q-value for the right action produces a positive input, and equal Q-values result in an input of 0 . In the case of equal Q-values, the action selection neuron randomly selects a left or right action. The action selection neuron also receives random decision input from the random action neuron. Receiving this input overrides both the spiking input from the output layer and the internally generated random decision of the action selection neuron.

5) *Weight Update*: The weight update module determines how much to adjust the Q-table values and sends this weight update trace, y_1 , to the output layer. The weight update equation used in this training algorithm is the same as the Q-learning update rule in Equation 1—with the learning rate scaled by a factor of $\frac{1}{8}$ to prevent the $\max_a Q(s_{t+1}, a)$ term from overflowing within the allotted 8 bits of precision.

TD_{target} is calculated by the target calculator process, *Tg*, using Equation 2. In order to determine the maximum Q-value of the current state-action pair, $\max_a Q(s_{t+1}, a)$, the neuron receives distinct inputs corresponding to the Q-values of the left and right actions. The maximum of these two inputs is computed using a simple function available in Lava microcode.

The target calculator neuron also receives the reward input r_{t+1} from a relay neuron, *Rw*₁, downstream of the CartPole neuron's reward output. For a positive reward value, r_{t+1} is

added to $\gamma \max_a Q(s_{t+1}, a)$, whereas for a negative reward value, only r_{t+1} is sent as output. This is because the expected maximum future reward following a failure state, $\max_a Q(s_{t+1}, a)$, is assumed to be 0 ; thus, TD_{target} simplified to $TD_{\text{target}} = r_{t+1} + 0$ in such cases. The reward input also serves as a gating signal for the neuron's activity. If no reward input is received, then the neuron produces no output. Otherwise, TD_{target} is sent to the update calculator layer, *Up*.

The update calculator process is responsible for performing the calculations shown in Equations 3 and 4. To do this, it receives three distinct types of input. The first input to the neuron consists of the summation of the TD_{target} value sent from the target calculator neuron and $-Q(s_t, a_t)$, which is transmitted by a gated relay neuron, *O*_t, used to store the state-action Q-values from time t . As a result, the quantity received at this input is $TD_{\text{target}} - Q(s_t, a_t)$. The neuron also receives learning rate, α , input from a relay neuron, α_1 . This input gates the activity of the update calculator neuron, as the learning rate is multiplied by the $TD_{\text{target}} - Q(s_t, a_t)$ term and $\frac{1}{8}$ to compute U . If no learning rate input is provided, the resulting value of U would be 0 . The third type of input received by the update calculator process is the action selection at time t from relay neuron *A*_t. The update calculator process consists of two neurons, which allow it to send different update values for each Q-value associated with a given state.

The final microcoded process to be discussed is the weight update trace process, *Wu*, which consists of two neurons. The first input received by both neurons in the weight update trace process is the output from the left update calculator neuron added to the existing Q-value for the leftward action associated with state s_t . This quantity can be represented as $U_0 + Q(s_t, a_0)$. The second input received by both neurons is the output from the right update calculator neuron added to the existing Q-value for the rightward action, or $U_1 + Q(s_t, a_1)$. Relay neuron *O*_t, downstream of the output layer, is used to store the Q-values output at time t . This neuron is connected to the weight update trace process via a one-to-one connection. Thus, for the third input, each weight update trace neuron individually receives a Q-value corresponding to state s_t and either action a_0 or a_1 . The updated Q-value is constrained to the 8-bit signed range using the equation:

$$Q^{\text{new}}(s_t, a_t) \leftarrow \min(\max(Q^{\text{new}}(s_t, a_t), -127), 126) \quad (12)$$

This range was chosen to be slightly narrower than the full 8-bit signed integer range $[-128, 127]$ to prevent potential overflow resulting from stochastic rounding. Once the updated Q-value is bounded, the weight update trace neuron checks if the resulting left and right Q-values are equal. If so, and if both saturate at 126 , then the updated Q-value is set to 126 and the opposing Q-value is adjusted to 125 to maintain a difference. The trace values y_1 are then computed as the difference between the new and previous Q-values. These are sent to the output layer to drive Hebbian learning according to the update Equations 9 and 10.

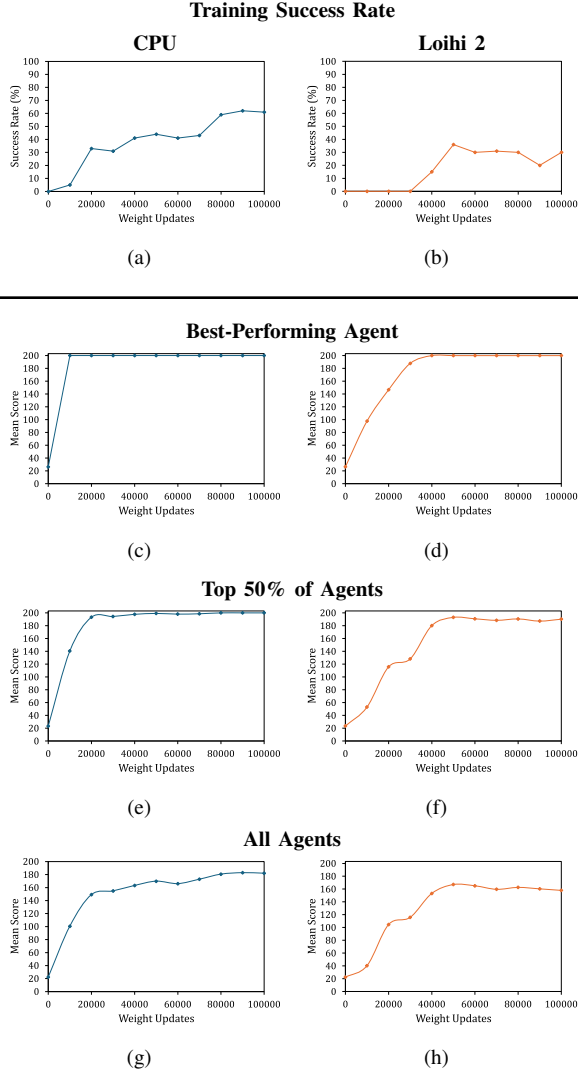


Fig. 4. Agent performance on CartPole-v0 after Q-learning on CPU and Loihi 2. **a**: percentage of CPU-trained agents scoring 195 or higher, **b**: percentage of Loihi 2-trained agents scoring 195 or higher, **c**: CPU-trained top-performing agent, **d**: Loihi 2-trained top-performing agent, **e**: CPU-trained top 50% of agents, **f**: Loihi 2-trained top 50% of agents, **g**: all CPU-trained agents, **h**: all Loihi 2-trained agents.

III. RESULTS

A. Agent Performance

To evaluate our neuromorphic Q-learning algorithm against a standard implementation, we trained 100 agents at each of 10 different weight update counts (10 000 to 100 000 in increments of 10 000) on both Loihi 2 and an Intel Core i7-10870H CPU, yielding 1 000 agents per platform. Both implementations used hyperparameters described in Table I. The resulting Q-table matrices were used for inference in the OpenAI Gym CartPole-v0 environment. In CartPole-v0, the agent’s score corresponds to the number of cumulative time steps during which the pole remains balanced with the cart in bounds. An agent is considered to have solved the CartPole-v0 task if it achieves a mean score of at least 195 over 100

trials, where each trial is capped at a maximum score of 200. Agent performance across both platforms is shown in Fig. 4—compared against pure random action selection, displayed as agents trained for 0 weight updates.

1) *CPU Training*: Visualizations of CPU-trained agents are shown in Figs. 1(g) and 1(h). The maximum success rate, defined as the proportion of agents able to solve CartPole-v0, achieved during CPU training was 62%, as shown in Fig. 4(a). This peak was reached at 90 000 weight updates and slightly decreased to 61% at 100 000 updates. The highest-performing CPU-trained agent attained a perfect mean score of 200 on CartPole-v0 after just 10 000 weight updates, as depicted in Fig. 4(c). The top 50% of CPU-trained agents achieved a mean score of 197.74 after 40 000 weight updates, as shown in Fig. 4(e). Their maximum average score reached a perfect 200 at 80 000 and 90 000 weight updates. Similarly, the average score across all CPU-trained agents peaked at 90 000 updates, reaching a value of 182.94, as shown in Fig. 4(g).

2) *Loihi 2 Training*: Visualizations of Loihi 2-trained agents are shown in Figs. 1(c) through 1(f), with visualizations during different stages of training shown in Figs. 1(i) through 1(m). The on-chip CartPole environment during training is shown in Figs. 1(n) through 1(r), with various levels of random action selection demonstrated in Figs. 1(s) and 1(t).

The maximum success rate achieved during training on Loihi 2 was 36%, as shown in Fig. 4(b). This peak occurred at 50 000 weight updates, after which the success rate plateaued near 30% from 60 000 updates onward. This success rate, while lower than the CPU’s 62%, represents the first demonstration of successful end-to-end RL training on neuromorphic hardware and achieves the same absolute number of successful agents in half the wall-clock time (see Section III-B). The higher CPU success rate likely stems from several factors: (1) 64-bit floating-point precision vs. 24-bit fixed-point on Loihi 2, (2) much higher trigonometric function accuracy (14–15 decimal places vs. 2–3), and (3) higher-quality random number generation.

Fig. 4(d) illustrates the performance of the best Loihi 2-trained agent, which successfully solved the CartPole-v0 environment after 40 000 weight updates, attaining a perfect mean score of 200 over 100 trials. As shown in Fig. 4(f), peak performance among the top 50% of agents was observed at 50 000 updates, at which point their mean score was 193.01. Fig. 4(h) shows the mean scores across all Loihi 2-trained agents. As with the other metrics, the peak average score was obtained at 50 000 weight updates and reached 167.03. Notably, CPU training peaks later (90 000 updates) than Loihi 2 (50 000 updates), suggesting the neuromorphic system may converge faster initially but reach a lower final performance ceiling due to precision limitations. Overall, the neuromorphic Q-learning algorithm achieved strong results in solving the CartPole-v0 task.

B. Performance Profiling

Table II shows performance profiling for Q-learning on Loihi 2 and the CPU. Measurements for the CPU were

TABLE II
BREAKDOWN OF PERFORMANCE PROFILING FOR Q-LEARNING TRAINING
AND INFERENCE ON LOIHI 2 AND THE CPU.

Task	Hardware	Power (W)			Latency (μ s)
		Static	Dynamic	Total	
Loihi 2 training	x86 cores	1.56	0.00	1.56	58.42
	neurocores (34)	0.22	0.02	0.25	
	whole board	1.78	0.02	1.80	
CPU training	Intel Core i7-10870H	31.23	12.00	43.22	162.54
Loihi 2 inference	x86 cores	1.56	0.00	1.56	12.86
	neurocores (12)	0.22	0.03	0.25	
	whole board	1.78	0.03	1.81	
CPU inference	Intel Core i7-10870H	31.64	11.63	43.27	128.55

Task	Hardware	Energy per action (μ J)		
		Static	Dynamic	Total
Loihi 2 training	x86 cores	90.51	0.00	90.51
	neurocores (34)	12.93	1.32	14.25
	whole board	103.44	1.32	104.75
CPU training	Intel Core i7-10870H	5 077.39	1 947.95	7 024.44
Loihi 2 inference	x86 cores	20.01	0.00	20.01
	neurocores (12)	2.86	0.36	3.22
	whole board	22.87	0.36	23.23
CPU inference	Intel Core i7-10870H	4 067.77	1 493.79	5 561.56

conducted using HWiNFO64 version 8.26-5730 on a Dell Alienware m15 R4 laptop running Microsoft Windows 10 Enterprise version 10.0.19045 Build 19045. The system was equipped with an Intel Core i7-10870H CPU operating at 2.20 GHz (2208 MHz), with 8 physical cores and 16 logical processors. Measurements for Loihi 2 were performed using Lava on Loihi version v0.6.0, running on Oheogulch board ncl-ext-og-05, generation N3B3, and recorded from on-chip energy and execution time probes. The Loihi 2 board contains both neurocores and x86 cores, so measurements are reported separately for each of these hardware components. Energy per action (bottom half of Table II) was calculated by multiplying power by latency.

Table II reveals that Loihi 2’s energy advantage stems from both lower power draw ($600\times$ during training) and faster execution ($2.8\times$ per weight update), compounding to yield over $1\,000\times$ energy efficiency for equivalent training success. Given that CPU training takes 2.8 times longer per weight update, Loihi 2 remains faster in terms of wall-clock time despite its lower success rate; requiring only 51% as much time to train the same number of successful agents.

We do not report GPU performance for CartPole Q-learning because a direct comparison to our neuromorphic implementation would have required a fully on-device environment identical to OpenAI Gym’s CartPole-v0 as well as integer-precision learning updates that match our fixed-point constraints (8-bit weights with 24-bit activations). This would require custom CUDA implementations of both the environment and the Q-updates, which is beyond the scope of this work and is unlikely to utilize the GPU efficiently for this iterative, constrained state space problem. As an example of the inefficiency of using

existing libraries, training on an NVIDIA Tesla V100S PCIe 32GB GPU via PyTorch using the OpenAI Gym CartPole-v0 environment with a 67% success rate resulted in three orders of magnitude higher energy consumption and an order of magnitude longer execution time for the GPU than Loihi 2.

IV. DISCUSSION

A. Training Algorithm Performance

This work demonstrates that neuromorphic hardware is not only capable of solving RL tasks like CartPole, but also can do so with energy efficiency over three orders of magnitude greater than traditional CPUs, while matching CPU-trained agent success in half the training time. Prior work has deployed reinforcement-learning policies and value networks on neuromorphic hardware, including Loihi-based robotic control and deep spiking Q-networks [23]–[25]. To the authors’ knowledge, the distinguishing contribution here is the fully on-chip, closed-loop Loihi 2 implementation in which the Q-learning update and the CartPole simulation execute on the neuromorphic processor without host computation between environment transitions. This model-free implementation therefore extends prior neuromorphic RL deployments from efficient policy execution toward a self-contained learning–environment loop.

B. Limitations

The neuromorphic implementation achieves 58% of the CPU’s success rate (36% vs. 62%) likely due to Loihi 2’s 24-bit fixed-point arithmetic compared to the CPU’s 64-bit floating-point arithmetic and reduced RNG performance in random action selection. Future work should investigate whether this gap can be narrowed through improved learning rate schedules adapted to fixed-point arithmetic, alternative Q-table representations, or improved RNG. While an LCG was necessary for use within the CartPole neuron, higher-quality generators such as Wichmann-Hill [50] could be used within the random action selection circuitry to improve state space exploration [48].

CartPole serves only as an initial benchmark for neuromorphic RL. More complex control tasks, such as unmanned aerial system (UAS) control [51], [52], would provide a more realistic assessment of real-world applicability and appear amenable to on-chip implementation, as suggested by our single-neuron representation of the CartPole environment. Demonstrating on-chip learning for such tasks would indicate that agents can be continuously adapted *in situ* without disrupting task execution, since our implementation already sustains over 17 kHz weight-update rates, comfortably above the 100–1 000 Hz sensing and actuation frequencies typical for quadcopter control [53].

C. Broad Implications

For the class of control tasks with discrete state–action spaces and real-time requirements—such as robotics, autonomous systems, and edge computing—this work demonstrates that neuromorphic RL offers a viable path toward $1\,000\times$ energy efficiency improvements. Real impact on AI sustainability will, however, require showing that such gains

extend to more complex benchmarks and ultimately to deployed applications.

Taken together with neuromorphic backpropagation [17], these results show that the synfire-gated synfire chain [18] coding framework can support a broad class of AI algorithms previously realized only on von Neumann architectures. Algorithms originally designed for globally-clocked processors can now be mapped onto asynchronous neuromorphic chips, suggesting that larger, more capable neuromorphic AI models are feasible and that this framework offers a plausible path toward energy-efficient, commercially relevant AI systems.

ACKNOWLEDGMENTS

We thank Alpha Renner for valuable discussions and acknowledge Intel Labs and the INRC community for providing access to the Loihi research chip and for discussions and support. This work has been approved for unlimited release by Los Alamos National Laboratory with number LA-UR-25-31518.

REFERENCES

- [1] B. Li and S. Gilbert, "Artificial intelligence awarded two nobel prizes for innovations that will shape the future of medicine," *npj Digital Medicine*, vol. 7, p. 336, 2024.
- [2] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for modern deep learning research," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 09, pp. 13 693–13 696, Apr. 2020. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/7123>
- [3] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green ai," *Commun. ACM*, vol. 63, no. 12, p. 54–63, Nov. 2020. [Online]. Available: <https://doi.org/10.1145/3381831>
- [4] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia *et al.*, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022.
- [5] L. F. W. Anthony, B. Kanding, and R. Selvan, "Carbontracker: Tracking and predicting the carbon footprint of training deep learning models," 2020. [Online]. Available: <https://arxiv.org/abs/2007.03051>
- [6] D. Amodei and D. Hernandez, "Ai and compute," *OpenAI*, 2018. [Online]. Available: <https://openai.com/blog/ai-and-compute/>
- [7] E. M. Bender, T. Geburu, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 610–623. [Online]. Available: <https://doi.org/10.1145/3442188.3445922>
- [8] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>
- [9] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks: training neural networks with low precision weights and activations," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 6869–6898, Jan. 2017.
- [10] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa *et al.*, "In-datacenter performance analysis of a tensor processing unit," *SIGARCH Comput. Archit. News*, vol. 45, no. 2, p. 1–12, Jun. 2017. [Online]. Available: <https://doi.org/10.1145/3140659.3080246>
- [11] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [12] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. F. Guerra, P. Joshi *et al.*, "Advancing neuromorphic computing with loihi: A survey of results and outlook," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 911–934, 2021.
- [13] Intel Labs, "Taking neuromorphic computing to the next level with Loihi 2 technology brief," Intel Labs, Intel Corporation, Tech. Rep., 2022. [Online]. Available: <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/neuromorphic-computing-loihi-2-brief.pdf>
- [14] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.
- [15] P. A. Merolla, J. V. Arthur, R. Alvarez-Icaza, A. S. Cassidy, J. Sawada, F. Akopyan *et al.*, "A million spiking-neuron integrated circuit with a scalable communication network and interface," *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [16] C. Mead, "Neuromorphic electronic systems," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1629–1636, 1990.
- [17] A. Renner, F. Sheldon, A. Zlotnik, L. Tao, and A. Sornborger, "The backpropagation algorithm implemented on spiking neuromorphic hardware," *Nature Communications*, vol. 15, no. 1, p. 9691, 2024. [Online]. Available: <https://doi.org/10.1038/s41467-024-53827-9>
- [18] Z. Wang, A. T. Sornborger, and L. Tao, "Graded, dynamically routable information processing with synfire-gated synfire chains," *PLOS Computational Biology*, vol. 12, no. 6, pp. 1–17, 06 2016. [Online]. Available: <https://doi.org/10.1371/journal.pcbi.1004979>
- [19] N. Frémaux, H. Sprekeler, and W. Gerstner, "Reinforcement learning using a continuous time actor-critic framework with spiking neurons," *PLoS Computational Biology*, vol. 9, no. 4, p. e1003024, 2013.
- [20] Z. Wang, C. Li, W. Song, M. Rao, D. Belkin, Y. Li *et al.*, "Reinforcement learning with analogue memristor arrays," *Nature Electronics*, vol. 2, pp. 115–124, 2019.
- [21] T. Dalgaty, N. Castellani, C. Turck, K.-E. Harabi, D. Querlioz, and E. Vianello, "In situ learning using intrinsic memristor variability via markov chain monte carlo sampling," *Nature Electronics*, vol. 4, pp. 151–161, 2021.
- [22] D. Vlasov, A. Minnekhanov, R. Rybka, Y. Davydov, A. Sboev, A. Serenko *et al.*, "Memristor-based spiking neural network with online reinforcement learning," *Neural Networks*, vol. 166, pp. 512–523, 2023.
- [23] G. Tang, N. Kumar, and K. P. Michmizos, "Reinforcement co-learning of deep and spiking neural networks for energy-efficient mapless navigation with neuromorphic hardware," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 6090–6097.
- [24] G. Tang, N. Kumar, R. Yoo, and K. Michmizos, "Deep reinforcement learning with population-coded spiking neural network for continuous control," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155, 2021, pp. 2016–2029.
- [25] M. Akl, Y. Sandamirskaya, F. Walter, and A. Knoll, "Porting deep spiking q-networks to neuromorphic chip loihi," in *International Conference on Neuromorphic Systems 2021*. Association for Computing Machinery, 2021.
- [26] A. Vitale, A. Renner, C. Nauer, D. Scaramuzza, and Y. Sandamirskaya, "Event-driven vision and control for UAVs on a neuromorphic chip," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 103–109.
- [27] L. Zanatta, A. Di Mauro, F. Barchi, A. Bartolini, L. Benini, and A. Acquaviva, "Directly-trained spiking neural networks for deep reinforcement learning: Energy efficient implementation of event-based obstacle avoidance on a neuromorphic accelerator," *Neurocomputing*, vol. 562, p. 126885, 2023.
- [28] S. F. Chevtchenko, Y. Bethi, T. B. Luderemir, and S. Afshar, "A neuromorphic architecture for reinforcement learning from real-valued observations," in *2024 International Joint Conference on Neural Networks (IJCNN)*, 2024, pp. 1–10.
- [29] D. Shin, H. Jo, H. Jang, Y. H. Jeong, Y. Jeong, J. Y. Kwak *et al.*, "Spike-based q-learning in a non-von neumann architecture," *Frontiers in Neuroscience*, vol. 20, p. 1738140, 2026.
- [30] C. J. Watkins, "Learning from delayed rewards," Ph.D. dissertation, University of Cambridge, 1989.
- [31] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [32] Q. Jiang, "Path planning method of mobile robot based on q-learning," *Journal of Physics: Conference Series*, vol. 2181, no. 1, p. 012030, Jan 2022. [Online]. Available: <https://doi.org/10.1088/1742-6596/2181/1/012030>
- [33] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "Safe, multi-agent, reinforcement learning for autonomous driving," 2016. [Online]. Available: <https://arxiv.org/abs/1610.03295>

- [34] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. A. Sallab, S. Yoganani *et al.*, “Deep reinforcement learning for autonomous driving: A survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2022.
- [35] J. Wang, W. Xu, Y. Gu, W. Song, and T. C. Green, “Multi-agent reinforcement learning for active voltage control on power distribution networks,” in *Proceedings of the 35th International Conference on Neural Information Processing Systems*, ser. NIPS ’21. Red Hook, NY, USA: Curran Associates Inc., 2021.
- [36] P. Yu, H. Zhang, Y. Song, Z. Wang, H. Dong, and L. Ji, “Safe reinforcement learning for power system control: A review,” *Renewable and Sustainable Energy Reviews*, vol. 223, p. 116022, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1364032125006951>
- [37] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.
- [38] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [39] D. Michie and R. A. Chambers, “Boxes: An experiment in adaptive control,” Ph.D. dissertation, University of Edinburgh, 1968.
- [40] J. S. Plank, C. P. Rizzo, C. A. White, and C. D. Schuman, “The cartpole application as a benchmark for neuromorphic computing,” *Journal of Low Power Electronics and Applications*, vol. 15, no. 1, 2025.
- [41] OpenAI, “Cartpole environment source code,” https://github.com/openai/gym/blob/master/gym/envs/classic_control/cartpole.py, 2024.
- [42] R. V. Florian, “Correct equations for the dynamics of the cart-pole system,” *Center for Cognitive and Neural Studies*, 2007. [Online]. Available: https://coneural.org/florian/papers/05_cart_pole.pdf
- [43] M. Matsumoto and T. Nishimura, “Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM Trans. Model. Comput. Simul.*, vol. 8, no. 1, p. 3–30, Jan. 1998. [Online]. Available: <https://doi.org/10.1145/272991.272995>
- [44] C. R. Harris, K. J. Millman, and S. J. van der Walt, “numpy.random.generator — numpy v2.1.dev0 manual,” 2024. [Online]. Available: <https://numpy.org/doc/2.1/reference/random/generator.html>
- [45] —, “Random sampling (numpy.random) — numpy v1.26 manual,” 2024. [Online]. Available: <https://numpy.org/doc/stable/reference/random/index.html#module-numpy.random>
- [46] D. H. Lehmer, “Mathematical methods in large-scale computing units,” in *Proceedings of the 2nd Symposium on Large-Scale Digital Calculating Machinery*. Harvard University Press, 1951, pp. 141–146.
- [47] cc65 Contributors, “cc65: A complete cross development package for 65(c)02 systems - rand.s,” 2000. [Online]. Available: <https://github.com/cc65/cc65/blob/06bb95d19788e3326738ee968b49dd11d18ca790/libsrc/common/rand.s>
- [48] S. Sinha, S. H. Islam, and M. S. Obaidat, “A comparative study and analysis of some pseudorandom number generator algorithms,” *SECURITY AND PRIVACY*, vol. 1, no. 6, p. e46, 2018. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spy2.46>
- [49] M. Muetsch, “Cartpole with q-learning: First experiences with openai gym,” 2020. [Online]. Available: <https://muetsch.io/cartpole-with-qlearning-first-experiences-with-openai-gym.html>
- [50] B. A. Wichmann and I. D. Hill, “Algorithm as 183: An efficient and portable pseudo-random number generator,” *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, vol. 31, no. 2, pp. 188–190, 1982. [Online]. Available: <http://www.jstor.org/stable/2347988>
- [51] O. A. Garcia A., D. C. Arana, E. S. Espinoza, I. R. Scola, L. R. Garcia Carrillo, and A. T. Sornborger, “Spiking neural network-based control of an unmanned aerial system implemented on a customized neural flight simulation environment,” in *2024 American Control Conference (ACC)*, 2024, pp. 3124–3129.
- [52] D. C. Arana, O. A. García A., L. R. Garcia Carrillo, I. Rubio Scola, E. S. Espinoza, and A. T. Sornborger, “Spiking neural network-based flight controller,” in *2024 Neuro Inspired Computational Elements Conference (NICE)*, 2024, pp. 1–8.
- [53] Kurbanov, Rashid, Litvinov, Maxim, Krivko, Stanislav, Zakharova, Natalia, and Kuprin, Alexey, “Justification of optimal parameters for quadcopter pid-controllers with frame sizes up to 150 mm,” *E3S Web Conf.*, vol. 431, p. 06014, 2023. [Online]. Available: <https://doi.org/10.1051/e3sconf/202343106014>